

## EXERCÍCIOS PROPOSTOS DE PROGRAMAÇÃO PARA O MIPS

Estes exercícios pode ser usados para gerar exemplos de programas a executar visa simulação do hardware do processador MIPS\_S\_withBRAMs.

### CONJUNTO DE EXERCÍCIOS PROPOSTOS 1

1. Subtrair uma constante a um vetor armazenado a partir da posição de memória cujo rótulo é *vet*. O número de elementos está armazenado na posição de memória cujo rótulo é *size*.
2. Fazer um algoritmo que lê um vetor de numerais (iniciando na posição de memória *vet*), calcula o número de elementos pares e ímpares e informa qual é o maior par e qual é o maior ímpar escrevendo nas posições *MaiorPar* e *MaiorImpar* os resultados. É possível programar duas versões deste programa, uma considerando o vetor como composto apenas de números naturais e uma considerando o vetor como composto apenas de números inteiros.
3. Escrever um programa para copiar um vetor de tamanho *vetsize* armazenado entre os endereços de memória com rótulos *inivet* e *fimvet* para os endereços cujos rótulos são *inicpvet* e *fimcpvet*. Assumir que as seguintes condições são verdadeiras:  $\text{valor}(\text{inivet}) < \text{valor}(\text{fimvet})$ ,  $\text{valor}(\text{inicpvet}) < \text{valor}(\text{fimcpvet})$ ,  $(\text{fimvet} - \text{inivet}) = (\text{fimcpvet} - \text{inicpvet}) = (\text{vetsize} - 1)$ .
4. Contar o número de posições de memória com conteúdo igual a uma constante, armazenada em uma posição de memória cujo rótulo é *CTE*, no vetor armazenado entre dois endereços marcados com os rótulos *inivet* e *fimvet*. O resultado deve ser armazenado na posição de memória *CTE\_vezes*.
5. Dados dois inteiros, *Int1* e *Int2*, armazenados, em posições de memória com estes rótulos, armazenar na posição de memória com rótulo *max* o  $\max(\text{Int1}, \text{Int2})$  (valor máximo entre *Int1* e *Int2*) e na posição de memória com rótulo *min* o  $\min(\text{Int1}, \text{Int2})$  (definido de forma similar ao max). Como *Int1* e *Int2* são inteiros, realize as comparações de forma correta.
6. Descobrir se um número *Num* é múltiplo de 4.
7. Dados dois números inteiros *Num1* e *Num2*, descobrir se *Num1* é múltiplo de *Num2*. Lembre-se que *Num1* pode ser um número positivo ou negativo.
8. Faça um algoritmo que calcula a divisão de dois números de 8 bits (armazenados nas posições de memória com rótulos *Num1* e *Num2*) através de subtrações sucessivas. Ao final do algoritmo a parte inteira da divisão deve estar na posição de memória com rótulo *quociente* e o resto na posição de memória com rótulo *resto*.

### CONJUNTO DE EXERCÍCIOS PROPOSTOS 2:

9. Multiplicar por somas sucessivas 2 inteiros positivos, armazenando o resultado em um inteiro longo. O multiplicando e o multiplicador devem estar armazenados nas posições de memória com rótulos *mcando* e *mcador*, respectivamente. O resultado deverá ser armazenado nas posições de memória com rótulos *multiplhigh* (a parte mais significativa do resultado) e *multiplow* (a parte menos significativa do resultado). O número de somas sucessivas deve ser o menor possível.
10. Fazer um programa que gere os *n* primeiros números da sequência de Fibonacci, e armazene esta subsequência em endereços consecutivos a partir da posição de memória com rótulo *fib*.
11. A transmissão de dados binários é o que viabiliza a existência de tecnologias como a Internet. A transmissão de dados a longas distâncias é uma tarefa muito propensa a erros, devido a efeitos ambientais externos (raios, interferências eletromagnéticas nas linhas de transmissão devido a equipamentos elétricos, raios cósmicos ou manchas solares que interferem nas transmissões de satélites etc.). Uma maneira de detectar erros de transmissão é acrescentar bits de controle ao dado transmitido, de forma que o receptor possa verificar a validade dos dados transmitidos. Um esquema simples de detecção de erros são as técnicas de bit de paridade. A idéia consiste em contar o número

de bits de um determinado valor e acrescentar um bit na mensagem que diz se a contagem destes valores na mensagem é par ou ímpar. Assim, pode-se imaginar alguns tipos básicos de cálculo de paridade: paridade de 0s ou paridade de 1s, paridade par ou paridade ímpar, paridade ativa em 0 ou paridade ativa em 1. Implemente um programa que recebe como entrada: (1)  $n$ , o número de bits de uma mensagem  $msg$ ; (2) a mensagem  $msg$ , de tamanho  $n$ . O programa deve produzir uma nova mensagem  $msg_{parity}$  com  $n+1$  bits, onde o bit mais significativo é a paridade da mensagem. Escolha o tipo de paridade a empregar e documente com comentários no programa as suas escolhas.

12. Escreva um programa para criar um vetor *novo*, a partir de dois vetores  $v1$  e  $v2$  de mesma dimensão  $n$ , segundo a relação estabelecida na equação abaixo. A interpretação da equação é: cada elemento de *novo*, na posição  $k$ , receberá o somatório dos valores máximos dos vetores  $v1$  e  $v2$ , entre 0 e  $k$ .

$$novo_k = \sum_{i=0}^k \max(v1_i, v2_i), \quad 0 \leq k < n$$

13. Seja dado um ponto inicial de coordenadas planas ( $x0, y0$ ) e uma sequência de  $n$  valores inteiros (o valor de  $n$  deve estar armazenado em uma posição de memória, e a sequência de valores deve estar armazenada como um vetor iniciando na posição de memória com rótulo *seq*). Implemente um algoritmo que desloque o ponto inicial alternadamente na horizontal e na vertical (iniciando com um deslocamento horizontal). Suponha que cada valor da sequência dá a magnitude do deslocamento seguinte. Considere que os deslocamentos são tais que o ponto atual e o imediatamente anterior circunscrevem um arco que avança no sentido anti-horário se o deslocamento atual for negativo, e no sentido horário se o deslocamento atual for positivo. Note que o deslocamento inicial pode ser em qualquer sentido horizontal, pois não há deslocamento anterior. Documente sua solução para indicar a escolha feita pelo seu programa.

Exemplo: dados de entrada [-6, -6, -4, 2, 3, 4] (ver Figura 1). Resultado -5 para  $x$  e -4 para  $y$ .  
Decisão: primeiro deslocamento para a esquerda se negativo, para a direita se positivo.

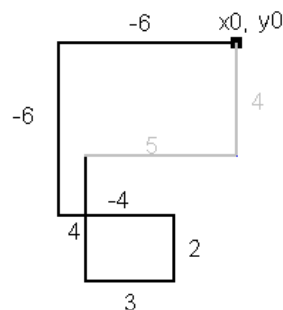


Figura 1 – Representação gráfica da entrada de dados [-6, -6, -4, 2, 3, 4].

14. Implemente um algoritmo simples de ordenamento crescente de números. Assuma que existe um vetor de entrada armazenado entre as posições de memória de rótulos *inivet* e *fimvet*, respectivamente. O resultado da execução deve gerar um ordenamento do vetor de entrada, armazenando este entre as posições de memória *inivetord* e *fimvetord*, respectivamente.
15. Faça um programa para ordenar 20 valores hexadecimais que estão armazenados em 10 bytes. Lembre-se que um byte pode ser visto como composto por dois *nibbles* (um *nibble* é uma sequência de exatamente 4 bits). O *nibble* menos significativo (o mais à esquerda no exemplo abaixo) deve conter o menor valor.
- Exemplo: Vetor de entrada: 0x34, 0xFA, 0xBC, 0x77, 0x1C  
Vetor ordenado: 0x13, 0x47, 0x7A, 0xBC, 0xCF
16. Escrever um algoritmo que calcule os primeiros  $n$  números primos e os armazene sequencialmente em um vetor, a partir da posição de memória cujo rótulo é *nprimos*.